

Protótipo de Assistente Virtual Baseado em RAG para Consulta às Normas da UFPI: Um Estudo de Caso

Guilherme T. Melo¹, Armando S. Sousa¹

¹Departamento de Computação – Universidade Federal do Piauí (UFPI)
Campus Universitário Ministro Petrônio Portella – Ininga
64.049-550 – Teresina – PI – Brasil

guilhermetorres@ufpi.edu.br, armando@ufpi.edu.br

Abstract. *Access to normative information in universities is often hindered by the complexity of legal texts. This work presents a virtual assistant based on Retrieval-Augmented Generation (RAG) to answer questions about UFPI's Resolution 177/2012. The architecture employs intelligent agents orchestrated via LangGraph and an exhaustive vector search strategy to mitigate hallucinations. Internal benchmarks demonstrated 100% retrieval success rate after engineering optimizations. Field experiments with students showed a 400% reduction in search time compared to manual methods, with a 98% perceived accuracy rate, validating the feasibility of Agentic RAG for academic support.*

Resumo. *O acesso à informação normativa na UFPI é dificultado pela extensão e complexidade dos textos jurídicos. Este trabalho apresenta um assistente virtual baseado em Retrieval-Augmented Generation (RAG) para a Resolução 177/2012. A arquitetura utiliza agentes inteligentes orquestrados via LangGraph e uma estratégia de busca vetorial exaustiva para mitigar alucinações. Testes internos demonstraram 100% de taxa de sucesso na recuperação após otimizações de engenharia. Experimentos com usuários evidenciaram uma redução de 400% no tempo de busca em comparação ao método manual, com 98% de eficácia percebida, validando a viabilidade de Agentic RAG para o suporte acadêmico.*

1. Introdução

A gestão acadêmica em Instituições de Ensino Superior (IES) é regida por um complexo arcabouço normativo que define os direitos, deveres e procedimentos da vida universitária. Na Universidade Federal do Piauí (UFPI)¹, o documento central que norteia essas relações é o **Regulamento Geral da Graduação referente as normas de funcionamento dos cursos de graduação da UFPI baseado na Resolução 177/2012**². Este documento atua como a base legal para todos os cursos de graduação da instituição, regulamentando desde o ingresso e matrícula até processos de avaliação, mobilidade acadêmica, conclusão de curso e jubilação.

Apesar de sua importância crítica para discentes e docentes, o acesso efetivo à Resolução 177/2012 enfrenta barreiras práticas e comportamentais. A dificuldade central

¹<https://www.ufpi.br>

²Regulamento Geral da Graduação da UFPI (Resolução 177/2012, atualizada 2023). Disponível em: Norma 177/2012

não reside apenas na complexidade textual, mas na dispersão de emendas e atualizações que frequentemente dificultam a localização da versão consolidada da norma, gerando incerteza sobre a vigência de determinadas regras. Somado a isso, observa-se uma baixa adesão dos discentes à consulta de fontes primárias extensas, resultando em uma cultura de desinformação ou na busca por canais informais. Como consequência, é comum que dúvidas simples sobre prazos ou procedimentos resultem na perda de direitos ou na sobrecarga das coordenações de curso com atendimentos repetitivos.

Simultaneamente, o avanço dos modelos de linguagem - *Large Language Model* (LLM), impulsionado por arquiteturas como o *Transformer* [Vaswani et al. 2017], revolucionou a capacidade das máquinas de processar e gerar texto em linguagem natural, permitindo a criação de assistentes conversacionais mais fluidos [Adamopoulou and Moussiades 2020]. No entanto, a aplicação direta de ferramentas como o ChatGPT³ em contextos normativos específicos enfrenta um desafio crítico: a "alucinação". Modelos genéricos, por não possuírem acesso direto aos documentos internos da UFPI, tendem a gerar respostas plausíveis, porém factualmente incorretas, o que é inadmissível em um cenário onde a precisão da regra é mandatória.

1.1. Problema de Pesquisa

O problema central abordado neste trabalho reside na lacuna entre a **necessidade de informação rápida e precisa** por parte dos alunos e a **complexidade de acesso** aos documentos oficiais. Os métodos tradicionais de busca (como a procura por palavras-chave em arquivos PDF) exigem que o usuário conheça a terminologia exata das normas de graduação, enquanto os assistentes de IA genéricos carecem de confiabilidade factual.

Diante disso, formula-se a seguinte questão de pesquisa: *Como desenvolver um assistente virtual capaz de interpretar a linguagem natural dos estudantes e recuperar respostas precisas na Resolução 177/2012, mitigando o risco de alucinações inerente aos modelos generativos?*

1.2. Objetivos

1.2.1. Objetivo Geral

Desenvolver e validar um protótipo de chatbot baseado na arquitetura RAG (*Retrieval-Augmented Generation*) [Lewis et al. 2020], utilizando orquestração de agentes inteligentes para responder a dúvidas sobre o regulamento de graduação da UFPI com alta fidelidade factual e linguagem acessível.

1.2.2. Ações Específicas

Para alcançar o objetivo geral, a pesquisa estrutura-se em etapas sequenciais que se iniciam pela organização da base de conhecimento da Resolução 177/2012, implementando um pipeline de engenharia de dados capaz de preservar a hierarquia semântica dos artigos, capítulos e seções para otimizar a recuperação. Em seguida, desenvolve-se a arquitetura

³O ChatGPT é um chatbot de inteligência artificial generativa desenvolvido pela OpenAI e lançado em novembro de 2022.

tura de Agentes Autônomos (*Agentic RAG*) baseada em grafos de decisão (LangGraph⁴), permitindo que o sistema decida dinamicamente o momento adequado para consultar a Resolução 177/2012. Concomitantemente, são avaliadas e otimizadas as estratégias de recuperação da informação [Setty et al. 2024], estabelecendo um comparativo entre a busca vetorial aproximada (ANN) [Indyk and Motwani 1998] e a busca por similaridade (kNN) [Bentley 1975], [Altman 1992] visando a eliminação de falhas de contexto [Muja and Lowe 2009]. A etapa final consiste na validação da eficácia da ferramenta por meio de testes de usabilidade, confrontando a eficiência da busca assistida por IA com os métodos manuais tradicionais.

1.3. Justificativa

No âmbito **social e institucional**, o projeto aborda a desinformação acadêmica decorrente não apenas da complexidade textual, mas principalmente da dificuldade de acesso à versão consolidada e atualizada da norma. Frequentemente, resoluções sofrem alterações e emendas que não são refletidas imediatamente no texto original disponível, induzindo ao erro. Somado a isso, observa-se uma baixa adesão dos discentes à leitura preventiva dos regulamentos e um desinteresse pela consulta direta à fonte primária, gerando uma cultura de busca reativa. O assistente atua como um facilitador que centraliza a informação atualizada e traduz o rigor jurídico, empoderando o estudante.

1.4. Organização do Trabalho

O restante deste documento está organizado da seguinte forma: A Seção 2 fundamenta os conceitos de PLN (Processamento de Linguagem Natural) e arquiteturas RAG. A Seção 3 analisa o estado da arte em chatbots acadêmicos. A Seção 4 detalha o método científico e o desenho experimental. A Seção 5 descreve a arquitetura do sistema, as decisões de engenharia e a evolução do protótipo (V1 para V2). Por fim, a Seção 6 apresenta a validação experimental e a Seção 7 conclui o trabalho com as considerações finais.

2. Referencial Teórico

Esta seção estabelece a base conceitual e tecnológica necessária para a compreensão e desenvolvimento de um chatbot educacional fundamentado na arquitetura de RAG. A exploração teórica inicia-se com os princípios de PLN, percorrendo a evolução das arquiteturas de redes neurais que culminaram nos Modelos de Linguagem do tipo LLM. Em seguida, são detalhados os mecanismos de representação vetorial, como os *embeddings*, que são cruciais para a capacidade de compreensão semântica das máquinas.

2.1. Processamento de Linguagem Natural

O PLN é o campo da inteligência artificial que capacita computadores a compreender, interpretar e gerar a linguagem humana, tornando-se essencial na era do *Big Data* para extrair valor de dados não estruturados e lidar com complexidades como ambiguidades e gírias [Jurafsky and Martin 2023]. Para isso, o campo opera em duas frentes principais: o *Natural language understanding* (NLU) [Semaan 2012] - Compreensão de Linguagem

⁴O LangGraph é uma biblioteca da LangChain para construir aplicações de IA com statefulness e ciclos. Ele permite criar fluxos de trabalho complexos onde agentes e ferramentas podem se comunicar de forma iterativa.

Natural - focada em decodificar a intenção do usuário, e o *Natural Language Generation* (NLG) [Reiter and Dale 1997] - Geração de Linguagem Natural - responsável por produzir respostas coerentes.

A integração dessas duas áreas transforma a interação humano-máquina de uma análise passiva para um diálogo ativo, servindo como a base para aplicações modernas como assistentes virtuais, motores de busca e chatbots avançados. O salto qualitativo que tornou essa evolução possível se apoia em dois pilares tecnológicos: uma arquitetura de rede neural revolucionária e os modelos de grande escala construídos sobre ela.

2.1.1. Arquitetura Transformer

A arquitetura *Transformer*, apresentada em 2017 com o artigo “*Attention Is All You Need*”, representou um marco para o PLN, tornando-se a base para modelos como GPT e BERT [Vaswani et al. 2017]. Ela superou as limitações de modelos sequenciais anteriores, como as RNNs (*Recurrent Neural Network*), ao introduzir o mecanismo de auto-atenção (*self-attention*), que processa todos os elementos de um texto simultaneamente. Em vez de analisar palavra por palavra, a auto-atenção permite que o modelo pese a importância de cada termo em relação a todos os outros, capturando relações de longo prazo e o contexto global de forma muito mais eficiente. Isso permite que a arquitetura modele tanto o significado quanto a estrutura da linguagem com uma performance e escalabilidade sem precedentes.

2.1.2. Large Language Model

Os LLMs são modelos de linguagem da vanguarda do PLN, consistindo em modelos massivos baseados na arquitetura *Transformer*, treinados em vastos volumes de dados textuais. Operando como sofisticados sistemas de previsão estatística, eles aprendem a prever a próxima palavra em uma sequência, internalizando padrões complexos de gramática, conhecimento de mundo e até capacidades de raciocínio.

Seu funcionamento, no entanto, revela um desafio central: por serem probabilísticos, os LLMs não recuperam fatos, mas geram as sequências de palavras mais prováveis, o que pode resultar em “alucinações” — informações fluentes, mas factualmente incorretas. Essa limitação fundamental impulsiona a necessidade de arquiteturas como a RAG, que buscam ancorar a geração do modelo em fontes de conhecimento externas e verificáveis, aumentando sua confiabilidade para aplicações como chatbots.

2.2. Embeddings e Representação Vetorial

Word Embeddings são representações numéricas que traduzem palavras, frases ou documentos em vetores densos dentro de um espaço multidimensional [Turian et al. 2010]. Diferente de métodos antigos como *one-hot encoding*, que tratam cada palavra como isolada, os *embeddings* capturam o significado semântico: termos com sentidos semelhantes são mapeados para vetores geometricamente próximos [Almeida and Xexéo 2019].

Essa propriedade transforma o problema de entender a relação entre palavras em um problema geométrico, onde a similaridade pode ser calculada matematicamente. Um

dos avanços mais notáveis nessa área foi a demonstração de que as relações semânticas entre palavras poderiam ser preservadas como relações algébricas entre seus vetores [Mikolov et al. 2013]. Modelos modernos, como os LLMs baseados em *Transformers*, geram *embeddings* contextuais, nos quais o vetor de uma palavra se adapta dinamicamente ao seu uso na frase, permitindo uma compreensão muito mais rica e precisa do significado, que é a base para técnicas avançadas como a busca semântica.

2.2.1. Busca Semântica

A busca semântica é uma técnica de recuperação de informação que foca no significado e na intenção de uma consulta, superando a limitação da correspondência exata de palavras-chave. Utilizando *embeddings*, ela entende que "conserto de veículo" é uma resposta relevante para "manutenção de automóveis" porque seus vetores são geometricamente próximos. O processo consiste em converter tanto os documentos de uma base de conhecimento quanto a consulta do usuário em vetores.

O passo crucial é então calcular a similaridade entre esses vetores. Embora existam diversas abordagens para medir a similaridade textual [Gomaa et al. 2013], a métrica mais robusta e utilizada para essa finalidade é a similaridade de cossenos. Em vez de medir a distância linear, ela calcula o cosseno do ângulo entre os vetores, focando exclusivamente em sua direção (o conteúdo semântico) e ignorando sua magnitude (o tamanho do texto).

O resultado varia de -1 a 1, onde 1 indica máxima relevância. Essa abordagem garante que um resumo curto e um artigo longo sobre o mesmo tópico sejam corretamente identificados como similares, tornando-se a ferramenta essencial que garante a precisão da busca semântica em arquiteturas como a RAG. A fórmula para este cálculo é definida como [Lahitani et al. 2016]:

$$\text{similaridade}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

2.3. Retrieval-Augmented Generation

O RAG, um framework introduzido por Lewis et al. (2020), foi projetado para aprimorar as capacidades dos modelos de linguagem do tipo LLM, conectando-os a uma fonte de conhecimento externa. Essa arquitetura resolve duas limitações críticas dos LLMs: o conhecimento estático, limitado aos dados de seu treinamento, e a tendência a gerar informações incorretas, fenômeno conhecido como "alucinação" [Lewis et al. 2020]. O RAG combina a busca precisa de sistemas de recuperação de informação com o poder de geração de texto dos LLMs, criando um sistema que baseia suas respostas em dados factuais e verificáveis.

2.3.1. Arquitetura Fundamental e Componentes

A operação de um sistema RAG segue um fluxo de duas etapas principais, conhecido como "recuperar e depois gerar" [Lewis et al. 2020]. Antes da execução, uma fase de

ingestão de dados é necessária, onde a base de conhecimento (documentos, artigos, etc.) é processada, dividida em pedaços menores (*chunks*), convertida em *embeddings* e indexada em um banco de dados vetorial. O fluxo principal então ocorre da seguinte forma:

- **Etapa de Recuperação (*Retrieval*):** Quando um usuário faz uma pergunta, o componente de recuperação (*retriever*) converte a consulta em um vetor e utiliza a busca semântica para encontrar os *chunks* de texto mais relevantes na base de conhecimento. A qualidade desta etapa é o gargalo de todo o sistema: se o contexto recuperado for irrelevante, a resposta final será igualmente ruim, seguindo o princípio de "*Garbage In, Garbage Out*".
- **Etapa de Geração Aumentada (*Augmented Generation*):** Os *chunks* de texto recuperados são combinados com a pergunta original, formando um prompt enriquecido. Este prompt é então enviado ao componente de geração (*generator*), um LLM. A tarefa do *generator* não é responder com base em sua memória interna, mas sim sintetizar uma resposta coesa, utilizando estritamente as informações factuais fornecidas pelo *retriever* [Lewis et al. 2020]. Ele atua como um tradutor, transformando os dados brutos em uma resposta conversacional e precisa.

Essa simbiose é o que torna o RAG poderoso: o *retriever* ancora o LLM na realidade com fatos verificáveis, enquanto o *generator* oferece uma interface de linguagem natural para esses fatos.

2.3.2. Técnicas de Otimização

Para maximizar a eficácia do RAG, diversas técnicas podem ser aplicadas ao pipeline, visando garantir a relevância do contexto fornecido. Uma das decisões fundamentais refere-se às Estratégias de *Chunking*, que determinam o método de divisão dos documentos originais. A escolha da estratégia impacta diretamente a qualidade da recuperação, variando desde divisões simples por tamanho fixo até métodos mais sofisticados, como o *chunking* semântico ou recursivo, que buscam preservar a estrutura lógica e o significado completo dos parágrafos.

Para superar a incompatibilidade de vocabulário entre o usuário e os documentos, utiliza-se a Expansão de Consulta (*Query Expansion*). Frequentemente, a pergunta do usuário não contém os mesmos termos técnicos presentes na base de conhecimento. Nesta técnica, um LLM é empregado para gerar sinônimos ou variações da pergunta original, enriquecendo a busca e aumentando a probabilidade de encontrar documentos relevantes (aumento do *recall*).

Por fim, uma estratégia eficaz para precisão é o *Re-ranking*. Esta técnica introduz uma segunda etapa de verificação após a recuperação inicial. Enquanto um primeiro modelo, mais rápido e menos preciso, recupera uma lista ampla de candidatos, um segundo modelo (geralmente um *cross-encoder*) reavalia e reordena essa lista com base na relevância real para a pergunta. Essa abordagem tem se mostrado uma das otimizações *zero-shot* mais eficientes para filtrar ruído antes da etapa de geração [Setty et al. 2024].

Essas técnicas, entre outras, transformam um sistema RAG básico em uma solução robusta e de alta performance, capaz de lidar com a complexidade e a variabilidade das consultas em linguagem natural.

2.4. Chatbots

Chatbots, ou assistentes conversacionais, são sistemas projetados para simular a conversação humana e fornecer respostas aos usuários em linguagem natural [Adamopoulou and Moussiades 2020]. Sua concepção teórica remonta a ideias seminais como o Teste de Turing, que propôs o desafio de uma máquina exibir comportamento inteligente indistinguível de um humano [Turing 2007]. As primeiras implementações, como o ELIZA, eram sistemas baseados em regras rígidas que seguiam fluxos pré-definidos [Weizenbaum 1966].

Hoje, impulsionados pelos avanços em Processamento de Linguagem Natural, eles evoluíram para agentes dinâmicos que utilizam Inteligência Artificial, especialmente os modelos de linguagem do tipo LLM. Essa evolução permite interações muito mais fluídas, mas o poder generativo dos LLMs traz o desafio das "alucinações" — a tendência de gerar informações factualmente incorretas. É neste ponto que a arquitetura RAG se torna fundamental, pois ancora as respostas do chatbot em uma base de conhecimento confiável, garantindo a precisão das informações fornecidas [Lewis et al. 2020].

2.4.1. Engenharia de Prompt e Controle de Alucinações

Para garantir que um chatbot RAG se comporte de maneira confiável, a Engenharia de Prompt é essencial. Trata-se da arte de construir instruções claras para o LLM, guiando-o para o resultado desejado. No contexto do RAG, o prompt combina instruções de sistema, o contexto recuperado e a pergunta do usuário.

O principal objetivo dessa engenharia é mitigar as alucinações. A técnica central para isso é a ancoragem em fontes (*grounding*), onde o prompt instrui o LLM a formular sua resposta exclusivamente com base nas informações fornecidas no contexto. O sucesso dessa abordagem depende criticamente da qualidade do texto recuperado, que deve conter a evidência necessária para fundamentar a resposta [Karpukhin et al. 2020]. Instruções como "Use apenas os seguintes trechos para responder. Se a resposta não estiver nos trechos, diga que não sabe." são cruciais.

Complementarmente, é vital permitir a "não resposta". O prompt deve incluir uma regra para que, se a informação não for encontrada, o chatbot admita que não possui a resposta, em vez de inventar uma. A necessidade de os modelos aprenderem a se abster de responder quando a evidência é insuficiente foi um avanço chave na área de compreensão de máquina, tornando os sistemas de perguntas e respostas significativamente mais robustos e confiáveis [Rajpurkar et al. 2016]. Essa abordagem transforma o LLM de um "sábio" propenso a erros em um "especialista" focado e confiável.

3. Trabalhos Relacionados

Para posicionar este projeto no panorama científico e técnico atual, esta seção analisa trabalhos correlatos em dois eixos principais. Primeiramente, são investigadas implementações de chatbots que aplicam a arquitetura RAG em domínios de conhecimento específicos, com foco especial no contexto acadêmico-normativo. Em seguida, é revisada a literatura técnica focada na otimização da etapa de recuperação, um componente crítico para a precisão de sistemas RAG.

3.1. Aplicação de RAG no Contexto Acadêmico-Normativo

Em um trabalho de grande relevância para o presente projeto, Silveira Júnior (2023) desenvolveu um chatbot para responder a perguntas sobre o regimento geral da Universidade Federal de Pelotas (UFPel). A metodologia empregada alinha-se fundamentalmente com a arquitetura de Geração Aumentada por Recuperação (RAG), utilizando a API do modelo GPT-3.5 da OpenAI⁵ como componente gerador e a busca semântica para a recuperação de contexto. O autor implementou um pipeline que envolve a segmentação (*chunking*) do documento oficial, a criação de *embeddings* vetoriais e o cálculo de similaridade de cossenos para identificar os trechos mais pertinentes à consulta do usuário. Notavelmente, o estudo também aplicou a engenharia de prompt para instruir o modelo a basear suas respostas estritamente no contexto fornecido, validando a viabilidade desta abordagem para garantir a factualidade e mitigar a geração de informações incorretas em um domínio de conhecimento específico [Silveira Júnior 2023].

Apesar do forte alinhamento metodológico, o presente trabalho busca aprofundar e expandir a abordagem de Silveira Júnior (2023), especialmente no que tange à robustez da etapa de recuperação e à avaliação do sistema. Enquanto o estudo anterior identificou como limitação a dificuldade em lidar com perguntas ambíguas, nosso projeto visa explorar técnicas avançadas de recuperação, como a busca vetorial exata e a filtragem estruturada de metadados para refinar a seleção de contexto antes da etapa de geração. Adicionalmente, este trabalho propõe uma análise que vai além de métricas de similaridade semântica automática como o BERTScore [Zhang et al. 2019], incorporando uma avaliação qualitativa com usuários reais focada na verificação da consistência factual e na utilidade prática das respostas.

3.2. Estratégias de Otimização para a Etapa de Recuperação em RAG

Aprofundando a análise sobre a otimização de sistemas RAG, Setty et al. (2024) investigam o impacto da etapa de recuperação na precisão de modelos de resposta a perguntas em domínios especializados, especificamente em documentos financeiros. O estudo argumenta que a principal limitação da performance de um sistema RAG não reside no poder gerativo do LLM, mas sim na qualidade do contexto que lhe é fornecido. Os autores identificam deficiências em pipelines RAG tradicionais, como a segmentação (*chunking*) uniforme, que desconsidera a estrutura semântica do documento, e a dependência excessiva da similaridade de cossenos aproximada, que nem sempre garante o retorno da informação mais relevante. O artigo avalia empiricamente técnicas de otimização *zero-shot*, demonstrando que melhorias no mecanismo de recuperação são essenciais para reduzir alucinações [Setty et al. 2024].

O presente trabalho se inspira diretamente nas conclusões de Setty et al. (2024), adotando a premissa de que a otimização da recuperação é fundamental. No entanto, enquanto a pesquisa citada se concentra no domínio financeiro, nosso projeto adapta essas estratégias ao domínio de documentos normativos universitários. O diferencial de nossa abordagem reside na implementação de um pipeline de recuperação determinístico (Exact kNN) em detrimento de buscas aproximadas, visando resolver o problema de "falsos negativos" na recuperação de artigos específicos, com uma avaliação final centrada na utilidade prática e na confiabilidade factual para a comunidade da UFPI.

⁵<https://openai.com/api/>

4. Metodologia

Esta seção detalha os procedimentos metodológicos adotados para a concepção, desenvolvimento e validação do assistente virtual. A pesquisa caracteriza-se como um estudo de caso aplicado, estruturado para resolver o problema de recuperação de informação no domínio normativo dos cursos de graduação da Universidade Federal do Piauí (UFPI).

4.1. Caracterização da Pesquisa

Quanto aos seus objetivos, a pesquisa é classificada como *exploratória e aplicada*, pois investiga o comportamento de novos modelos de IA em tarefas de interpretação de normas visando a entrega de um artefato de software funcional [Gil 2002]. A abordagem é *mista (quali-quantitativa)*, combinando métricas de desempenho do sistema com a análise da percepção dos usuários.

4.2. Objeto de Estudo e Corpus

O corpus documental restringe-se ao Regulamento Geral da Graduação da UFPI (Resolução 177/2012), escolhido pela sua alta densidade informacional e relevância para a comunidade acadêmica.

4.3. Procedimentos Metodológicos

4.3.1. Fase 1: Construção do Conjunto de Referência (*Golden Dataset*)

Para validar o sistema, elaborou-se um conjunto de dados de referência (*Golden Dataset*) com 20 pares de perguntas e respostas. Esta técnica baseia-se em práticas de avaliação de sistemas de QA (*Question Answering*), onde um gabarito humano serve como verdade fundamental (*ground truth*) para aferir a precisão [Rajpurkar et al. 2016].

As perguntas utilizam linguagem natural informal para simular o comportamento real dos discentes. Os tópicos foram selecionados com base na recorrência de dúvidas comuns nas coordenações de curso, abrangendo eixos críticos como: Matrícula, Avaliação, Frequência e Láurea Acadêmica. O conjunto completo de questões e gabaritos utilizados para a validação está disponível publicamente no repositório do projeto⁶.

4.3.2. Fase 2: Desenvolvimento Iterativo e Seleção de Modelos

O desenvolvimento priorizou a viabilidade de implementação, testando modelos como GPT-4o-mini⁷, GPT-5-mini⁸ e Gemini 2.5 Flash⁹. O critério de seleção foi o equilíbrio entre custo computacional e capacidade de raciocínio, resultando na escolha do GPT-5-mini para a versão final (V2) devido à sua estabilidade na mitigação de alucinações.

⁶https://github.com/guilhermetorresm/tcc-chatbot-ufpi/blob/main/data/questoes_para_avaliar_o_agente.md

⁷<https://openai.com/pt-BR/index/gpt-4o-mini-advancing-cost-efficient-intelligence>

⁸<https://platform.openai.com/docs/models/gpt-5-mini>

⁹<https://ai.google.dev/gemini-api/docs/models>

4.3.3. Fase 3: Experimento de Campo (Prova de Conceito)

A validação final consistiu em um estudo de caso preliminar para prova de conceito, adotando uma abordagem *intra-sujeitos* (*within-subjects*), onde os mesmos participantes experimentam ambos os cenários (manual e assistido), permitindo a comparação direta de desempenho [Wohlin et al. 2012]. É importante ressaltar que este experimento tem caráter exploratório, visando demonstrar a viabilidade técnica e a eficácia percebida do artefato em um cenário controlado, não constituindo uma validação estatística formal em larga escala.

Protocolo. Uma amostra de 10 discentes do sexto período do curso de graduação em Ciência da Computação da UFPI (campus Ministro Petrônio Portela) respondeu a 5 questões utilizando o arquivo PDF (Resolução 177/2012) e, em seguida, usando o Chatbot do protótipo do assistente virtual. *Métricas.* Avaliou-se a taxa de acerto (Eficácia) e a satisfação do usuário (Qualidade de Uso). O formulário para coleta de dados está disponível no seguinte link¹⁰.

4.4. Considerações Éticas

Em conformidade com a Lei Geral de Proteção de Dados (LGPD) [Brasil 2018], o protótipo não armazenou dados pessoais identificáveis, garantindo o anonimato dos voluntários.

5. Especificação e Implementação do Protótipo

Esta seção descreve a arquitetura técnica, as decisões de engenharia de dados e os detalhes de implementação do assistente virtual. O desenvolvimento do sistema partiu de uma base arquitetural de código aberto (*starter kit*) para microserviços de agentes¹¹. Sobre esta infraestrutura, foi implementado um novo agente especializado, desenvolvido em conformidade com os padrões de projeto do *framework* original, ao qual foram integradas ferramentas (*tools*) personalizadas criadas especificamente para a interação com a base normativa (Resolução 177/2012) da UFPI.

Visando contribuir para a reprodutibilidade científica e a melhoria contínua da ferramenta, todo o código-fonte desenvolvido foi disponibilizado publicamente em repositório aberto¹².

5.1. Arquitetura do Sistema

A solução foi estruturada em três camadas lógicas distintas, ilustradas na Figura 1, garantindo a separação de responsabilidades entre a interface, o processamento e a persistência de dados.

¹⁰Formulário de Coleta de dados: https://docs.google.com/forms/d/e/1FAIpQLSd95QR_Qfju4uXXStC3ns4phGSpN3HqDJVEhxMnjmxfybnyww/viewform?usp=sharing&oid=107949035426461151528

¹¹Projeto base: <https://github.com/JoshuaC215/agent-service-toolkit>

¹²Repositório do Projeto: <https://github.com/guilhermetorresm/tcc-chatbot-ufpi>

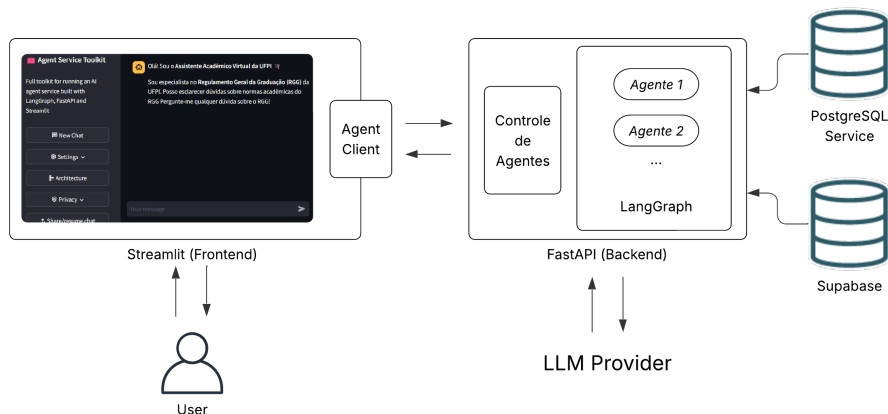


Figure 1. Arquitetura básica do assistente virtual (Frontend, Backend e Banco de Dados).

5.1.1. Interface (Frontend)

Implementada em *Streamlit*¹³, a interface atua como um cliente agnóstico, responsável exclusivamente pela interação com o usuário. Para garantir a continuidade do diálogo, o sistema gerencia a persistência do contexto conversacional através da criação de identificadores únicos de sessão (UUID v4). A experiência do usuário é otimizada através da renderização de respostas via *streaming*, exibindo o texto token a token para minimizar a latência percebida. Adicionalmente, a interface integra mecanismos de avaliação qualitativa (*Human-in-the-loop*) através de um *widget* de feedback, e promove a transparência algorítmica ao exibir os logs de execução das ferramentas (*Tool Calls*), permitindo a auditoria do fluxo de raciocínio do agente em tempo real.

A Figura 2 apresenta a tela principal do protótipo do assistente virtual, onde os principais componentes de interação estão numerados para detalhamento funcional.

Os elementos destacados na figura correspondem às seguintes funcionalidades:

1. **New Chat:** Botão que criará um novo chat.
2. **Settings:** Botão para abrir as configurações e possibilita escolher o agente para o chat, qual modelo de LLM do agente e se a saída deve ser via stream ou a padrão.
3. **Architecture:** Botão para abrir uma imagem da arquitetura geral do sistema.
4. **Privacy:** Botão que exibe a política de privacidade.
5. **Share/resume chat:** Botão para compartilhar o link do chat atual.
6. **Caixa de texto do Usuário:** Pergunta feita pelo usuário.
7. **Tool Call:** Ferramentas utilizadas pelo agente na pergunta, que ao clicar pode ser visto os parâmetros utilizados e a resposta da *tool*.
8. **Caixa de texto do Agente:** Resposta do agente para a pergunta.
9. **Avaliação:** Estrela para avaliação da resposta do agente.
10. **Campo de pergunta:** Caixa de texto para o usuário colocar sua pergunta.

¹³<https://streamlit.io/>

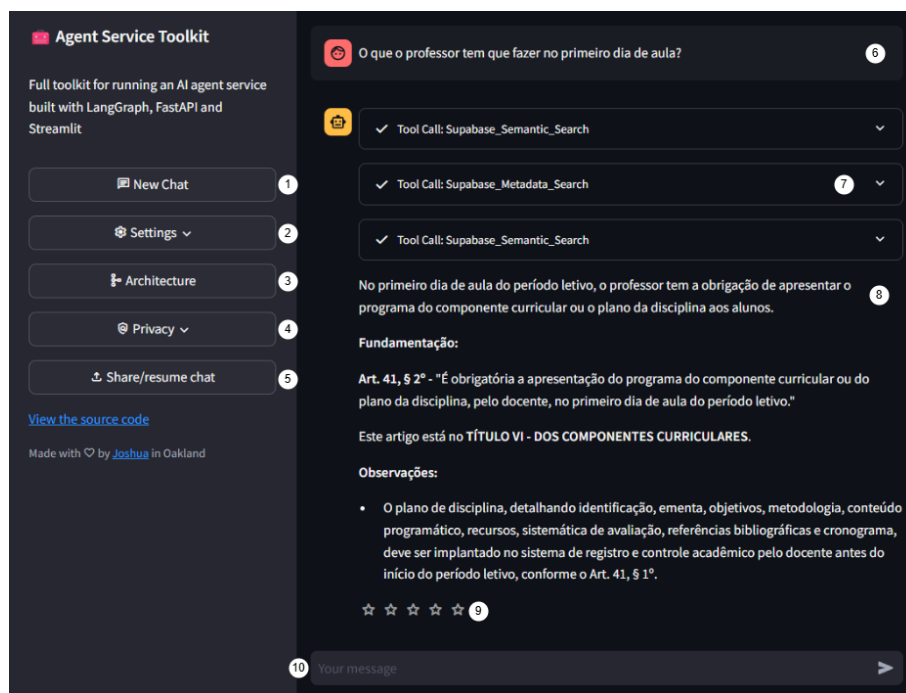


Figure 2. Visão geral da interface do assistente com destaque para os componentes funcionais.

5.1.2. Backend (API)

O núcleo de processamento foi desenvolvido em *FastAPI*¹⁴, expondo uma interface RESTful que atua como orquestradora entre a camada de apresentação e o núcleo de inteligência. O serviço foi projetado para suportar diferentes modalidades de interação, gerenciando a geração de respostas tanto em tempo real via *Server-Sent Events* (streaming) quanto de forma síncrona. Adicionalmente, a API centraliza o controle de estado, manipulando a recuperação do histórico de conversas e o registro de métricas de avaliação e feedback no sistema de observabilidade. A documentação técnica detalhada dos *end-points* e contratos de interface está disponível no repositório do projeto¹⁵.

5.1.3. Núcleo de Inteligência (*LangGraph*)

Diferente de cadeias lineares, o controle de fluxo conversacional foi implementado como um grafo de estados finito utilizando a biblioteca *LangGraph*¹⁶. Esta abordagem permite a execução de ciclos de raciocínio, onde o agente pode decidir iterativamente se precisa de mais informações, consultar ferramentas ou gerar a resposta final, conforme ilustrado na Figura 3.

¹⁴<https://fastapi.tiangolo.com/>

¹⁵https://github.com/guilhermetorresm/tcc-chatbot-ufpi/blob/main/data/backend_doc.md

¹⁶<https://www.langchain.com/langgraph>

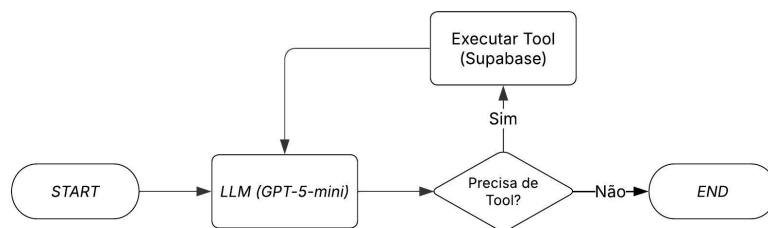


Figure 3. Grafo de decisão do agente inteligente (Fluxo LangGraph).

5.2. Engenharia de Dados e Pipeline de Ingestão

Para garantir que o modelo (LLM) compreendesse corretamente a estrutura hierárquica da Resolução 177/2012, optou-se por não utilizar extração de texto bruto. O processo seguiu um pipeline rigoroso de estruturação (Figura 4).

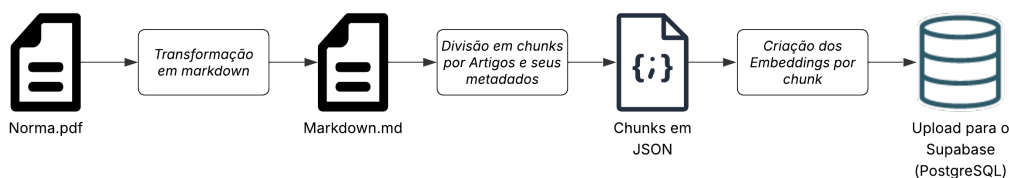


Figure 4. Pipeline de ingestão e processamento da Resolução 177/2012.

5.2.1. Conversão e Preservação de Hierarquia

O documento PDF original (Resolução 177/2012) foi convertido para o formato *Markdown* através de um processo assistido por LLM e revisão manual. Esta etapa foi crítica para preservar a semântica visual do regulamento, permitindo que o sistema diferencie o peso de um "Capítulo" em relação a um "Parágrafo Único".

5.2.2. Estratégia de Chunking Híbrido (Contribuição Original)

A segmentação dos dados (*chunking*) constitui uma das principais contribuições originais deste trabalho. Diferente das abordagens convencionais de RAG, que segmentam textos arbitrariamente por contagem de caracteres ou parágrafos fixos (o que frequentemente quebra o contexto jurídico), esta pesquisa implementou uma segmentação semântica baseada na unidade indivisível da norma: o **Artigo**.

Foi desenvolvido um script de segmentação (`chunks_v2.py`¹⁷) que isola cada artigo como um vetor independente, mas o enriquece com metadados hierárquicos (Capítulo, Seção). Essa estratégia garante a integridade hermenêutica da norma durante a recuperação, assegurando que o modelo compreenda o contexto completo mesmo ao recuperar um artigo isolado.

¹⁷https://github.com/guilhermetorresm/tcc-chatbot-ufpi/blob/main/code/scripts/chunks_v2.py

5.3. Modelagem do Agente e Grafo de Estados

A lógica do assistente foi implementada utilizando o framework *LangGraph*, modelada como um grafo cíclico de decisão. Conforme ilustrado na arquitetura do agente, o fluxo de execução opera através de um loop de raciocínio:

1. *Nó do Modelo (LLM)*: O ciclo inicia-se com o modelo (GPT-5-mini) recebendo o estado atual da conversa (histórico + última pergunta).
2. *Aresta Condicional (Precisa de Tool?)*: O sistema analisa a saída gerada pelo modelo.
 - **Caminho "Sim"**: Se o modelo invocar uma ferramenta (*Tool Call*), o fluxo é direcionado para o Nó de Ferramenta.
 - **Caminho "Não"**: Se o modelo gerar uma resposta textual direta sem invocar ferramentas, o fluxo é direcionado para o Fim (*END*), entregando a resposta ao usuário.
3. *Nó de Ferramenta*: Executa a busca no Supabase conforme os parâmetros definidos pelo modelo e, crucialmente, **retorna o resultado da busca para o Nó do Modelo**. Isso permite que o LLM processe a nova informação e decida novamente se precisa de mais dados ou se já pode sintetizar a resposta final.

Essa estrutura cíclica permite que o agente trate interações de forma dinâmica. Diante de uma entrada trivial (ex: "*Bom dia*"), o modelo não invoca ferramentas na primeira passagem, seguindo imediatamente para o encerramento (baixa latência). Já para consultas complexas (ex: "*Posso faltar quantas vezes?*"), o modelo entra no loop de execução, consulta a norma e, ao receber os dados, gera a resposta final na iteração seguinte.

A Figura 5 detalha um trace de execução onde esse ciclo ocorre.

5.3.1. Ferramentas Disponíveis (*Tools*)

O agente foi equipado com duas ferramentas especializadas:

- *Vector Search Tool*: Realiza uma busca por similaridade semântica para encontrar conceitos abstratos.
- *Metadata Search Tool*: Realiza filtragens exatas baseadas na estrutura da norma (ex: "Buscar especificamente o Artigo 64").

5.4. Arquitetura Cognitiva e Engenharia de Prompt

A confiabilidade do assistente foi assegurada através de uma engenharia de prompt avançada, onde as instruções atuam como um algoritmo determinístico que rege o comportamento do modelo. O *System Prompt* (disponível no repositório do projeto¹⁸) transforma o LLM de um gerador probabilístico num orquestrador de investigação rigoroso, definindo protocolos estritos para a manipulação da base de conhecimento.

O núcleo operacional reside na integração entre Recuperação Iterativa e Contextualização Hierárquica. O prompt impõe um loop de execução obrigatório com

¹⁸https://github.com/guilhermetorresm/tcc-chatbot-ufpi/blob/main/data/prompt_v2.md

Cenário: Usuário pergunta sobre direitos em caso de doença.

1. Entrada (START):
"Perdi a prova final porque estava internado. Tenho direito a refazer?"

2. Nó do Modelo (Iteração 1):
 → Raciocínio Interno: A pergunta requer verificação da norma sobre "segunda chamada".
 → Saída: **Tool Call** (Invocar ferramenta de busca).

3. Decisão Condicional:
 → Precisa de Tool? **Sim**.

4. Nó de Ferramenta:
 → Execução: `Vector_Search(query="segunda chamada justificativa doença")`
 → Resultado: Retorna o texto do *"Art. 125... direito à segunda chamada..."* para o modelo.

5. Nó do Modelo (Iteração 2):
 → Entrada: Resposta da Tool - Artigo 125 recuperado.
 → Raciocínio Interno: Tenho a informação necessária.
 → Saída: Resposta final em linguagem natural.

6. Decisão Condicional:
 → Precisa de Tool? **Não**. → **END**.

Figure 5. Trace de execução do agente demonstrando o ciclo de decisão e uso de ferramenta.

Refinamento Progressivo de Consulta, onde o agente realiza múltiplas buscas degradando a especificidade dos termos para resolver incompatibilidades de vocabulário e maximizar o *Recall*. Simultaneamente, para mitigar a perda de contexto de vetores isolados, o sistema obriga o uso complementar de ferramentas: ao localizar um tópico via busca semântica, o agente deve recuperar a estrutura completa (capítulos ou seções) via metadados, garantindo acesso a parágrafos e exceções adjacentes antes de processar a informação.

Por fim, a segurança é garantida por camadas de **Guardrails de Escopo e Raciocínio Oculto**. O fluxo inicia-se com uma classificação de intenção, rejeitando automaticamente perguntas fora do domínio normativo (como questões administrativas). Durante a formulação da resposta, o agente executa uma cadeia de pensamento (*Chain-of-Thought*) interna para verificar evidências em *logs* ocultos, entregando ao usuário final apenas a resposta sintetizada e estritamente ancorada (*Grounded*) nos vetores recuperados, eliminando a superfície para alucinações.

5.5. Infraestrutura de Dados

A persistência foi dividida estrategicamente para otimizar a recuperação estruturada:

5.5.1. Base de Conhecimento e Estrutura Normativa (Supabase)

Utilizou-se o **Supabase**¹⁹ com a extensão `pgvector`. Além de armazenar os vetores para busca semântica, o esquema do banco de dados foi desenhado para refletir a árvore hierárquica da Resolução 177/2012.

Cada registro na tabela de documentos possui colunas dedicadas para os metadados estruturais: `titulo_nome`, `capitulo_nome`, `secao_nome` e `artigo_numero`. Essa modelagem relacional permite que o agente execute consultas híbridas, combinando a flexibilidade da busca vetorial (conteúdo) com a precisão dos filtros hierárquicos (estrutura), garantindo que um artigo nunca seja recuperado fora do seu contexto original.

5.5.2. Memória de Conversação

A persistência do estado da sessão (*Checkpointing*) foi mantida em uma instância isolada do PostgreSQL via Docker. Isso garante que o contexto da conversa seja preservado entre interações sem onerar a base de conhecimento principal.

5.6. Evolução e Otimização do Protótipo (V1 para V2)

O desenvolvimento do sistema seguiu uma abordagem iterativa. Após os testes iniciais com a versão V1, foram identificados gargalos na precisão da recuperação e na autonomia do agente. Duas alterações arquiteturais críticas foram implementadas na versão V2 para solucionar esses problemas:

5.6.1. Refatoração das Ferramentas (Decomposição de Responsabilidades)

Na primeira versão (V1), o agente dispunha de uma única ferramenta monolítica que tentava realizar simultaneamente a busca vetorial e a filtragem por metadados baseada na entrada do usuário. Testes qualitativos demonstraram que essa sobrecarga cognitiva induzia o modelo a erros, frequentemente ignorando filtros importantes ou confundindo parâmetros.

Na versão V2, aplicou-se o princípio da separação de responsabilidades, dividindo a funcionalidade em duas ferramentas discretas:

- *Vector Search Tool*: Especializada exclusivamente em similaridade semântica para perguntas conceituais.
- *Metadata Filter Tool*: Especializada em buscas exatas (*Lookup*) quando o agente identifica uma referência explícita (ex: "Artigo 15").

Essa granularidade permitiu que o grafo de decisão do agente escolhesse a estratégia mais adequada para cada tipo de pergunta, reduzindo a ambiguidade.

5.6.2. Otimização da Recuperação: Busca Vetorial Exaustiva

A mudança mais impactante na performance do sistema ocorreu na camada de persistência vetorial. Inicialmente, a tabela de documentos no Supabase utilizava um índice

¹⁹<https://supabase.com/>

IVFFlat (*Inverted File Flat*), configurado para realizar uma Busca Aproximada de Vizinhos (ANN - *Approximate Nearest Neighbor*).

Durante a fase de validação, observou-se que a taxa de recuperação (*Recall*) era inconsistente para o volume de dados do projeto. O algoritmo IVFFlat funciona particionando o espaço vetorial em *clusters* para acelerar a busca. Dado o tamanho reduzido do *dataset*, essa fragmentação gerava "efeitos de borda": documentos relevantes situados na fronteira de *clusters* vizinhos não visitados eram ignorados pelo índice.

Para corrigir isso, na versão V2, optou-se por remover a indexação aproximada, forçando o banco de dados a realizar uma **Busca Vetorial Exaustiva** (também conhecida como *Flat Scan* ou *Brute-force kNN*).

Justificativa Teórica. Embora a busca exaustiva tenha complexidade computacional linear de $O(N)$, para o corpus da Resolução 177/2012 (aproximadamente 344 vetores), o custo é desprezível em hardware moderno. Essa decisão arquitetural eliminou a probabilidade de erro algorítmico na recuperação, garantindo que 100% da base de conhecimento seja varrida a cada consulta, maximizando a taxa de sucesso na recuperação.

5.7. Validação dos Ajustes de Engenharia

A definição da arquitetura final (V2) não se baseou apenas em métricas de desempenho isoladas, mas na interação entre a capacidade de recuperação do banco de dados e o comportamento de segurança do modelo de linguagem.

Para validar essa sinergia, realizou-se um *benchmark* interno utilizando o *Golden Dataset* (20 questões), comparando os modelos bases na arquitetura antiga (V1) contra a solução refinada (V2). A Tabela 1 sintetiza esses resultados em termos de taxa de sucesso.

Table 1. Comparativo de sucesso na recuperação entre a arquitetura base (V1) e a proposta (V2)

Métrica	GPT-4o-mini (V1 - Base)	Gemini 2.5 (V1 - Base)	GPT-5-mini (V1 - Base)	GPT-5-mini (V2 - Final)
Estratégia	<i>Aprox. (ANN)</i>	<i>Aprox. (ANN)</i>	<i>Aprox. (ANN)</i>	<i>Exaustiva</i>
Sucesso (Acerto)	55% (11/20)	50% (10/20)	60% (12/20)	100% (20/20)
Não Encontrou	25% (05/20)	40% (08/20)	40% (08/20)	0% (00/20)
Alucinações	20% (04/20)	10% (02/20)	0% (00/20)	0% (00/20)

5.7.1. Análise Integrada: Engenharia e Segurança (*Safety First*)

A interpretação dos dados da Tabela 1 revela dois fenômenos distintos que justificam a configuração final do sistema:

1. **O Impacto da Recuperação (Engenharia):** Observa-se que, na V1, todos os modelos sofreram com altas taxas de falha em encontrar a informação ("Não Encontrou" variando de 25% a 40)
2. **O Critério de Honestidade (Seleção do Modelo):** Comparando os modelos na V1, nota-se um comportamento divergente diante da falta de informação. O **GPT-4o-mini**, ao não encontrar o documento, tendeu a "alucinar" uma resposta (20%

de erro), o que é crítico em contextos normativos. Em contrapartida, o **GPT-5-mini** demonstrou um comportamento mais seguro: quando a busca falhava, ele preferia admitir que não sabia (40% de "Não Encontrou") a inventar dados (0% de alucinação).

Conclusão: A escolha do GPT-5-mini para a versão final não se deu apenas pela sua taxa de acerto bruta, mas pela sua "honestidade algorítmica". Ao combinar um modelo altamente resistente a alucinações com uma infraestrutura de dados que não falha na recuperação (V2), o sistema alcançou a robustez necessária para o ambiente acadêmico, atingindo 100% de precisão sem comprometer a confiabilidade das informações.

6. Resultados e Discussão

A avaliação do sistema foi conduzida para verificar duas hipóteses centrais: (1) a otimização da arquitetura de dados minimiza drasticamente a ocorrência de alucinações (validada nos testes internos de engenharia) e (2) o uso de interfaces conversacionais reduz a barreira de acesso à informação normativa para o estudante.

6.1. Execução do Experimento com Usuários

O experimento de campo contou com a participação de 10 discentes do sexto período do curso de Ciência da Computação da UFPI (campus Ministro Petrônio Portela), submetidos a um protocolo de teste *intra-sujeitos* em duas etapas. Na primeira fase, os participantes buscaram respostas para 5 questões sobre a Resolução 177/2012 que abordam temas frequentemente pesquisados pelos alunos utilizando o método tradicional (leitura do PDF oficial). Na segunda fase, repetiram a tarefa utilizando o assistente virtual desenvolvido (Versão V2 com GPT-5-mini e Busca Exata).

Table 2. Perguntas Seleccionadas sobre a Resolução 177/2012 da UFPI

Pergunta	Resposta Resumida
1. Como a UFPI calcula minha "média geral" (o tal do IRA)?	O Índice de Rendimento Acadêmico (IRA) é calculado considerando as notas obtidas nas disciplinas, seus respectivos créditos e o histórico de aprovações e reprovações, atribuindo maior peso às disciplinas com carga horária maior e penalizando reprovações.
2. Qual é a frequência mínima que eu preciso ter nas aulas para não ser reprovado por falta?	O estudante deve possuir, no mínimo, 75% de frequência na carga horária total da disciplina para evitar reprovação por faltas.
3. Fui reprovado por média, mas minha nota parcial foi 5,0. Eu tenho direito ao Exame Final?	Sim. O estudante que obtém média parcial entre 4,0 e 6,9 tem direito ao Exame Final, conforme regras gerais do regulamento de graduação.
4. Qual é a nota mínima que eu preciso para ser aprovado depois de realizar o Exame Final?	Para ser aprovado, a média aritmética entre a média parcial e a nota do Exame Final deve ser maior ou igual a 5,0.
5. Se eu for impedido de participar de uma avaliação (prova), posso requerer a oportunidade de fazer a segunda chamada?	Sim. Caso seja impedido de realizar avaliação por motivo justificável, o estudante pode solicitar segunda chamada, mediante apresentação de justificativa no prazo estabelecido pelo regulamento.

6.2. Análise de Eficiência e Satisfação

Os resultados obtidos demonstram uma disparidade significativa na eficiência operacional entre os dois métodos.

6.2.1. Ganho de Tempo e Redução de Esforço

A observação assistida durante os testes permitiu estabelecer uma métrica temporal média para a resolução das questões. Enquanto a busca manual no PDF exigiu, em média, cerca de **120 segundos (2 minutos)** por questão — demandando leitura de múltiplos artigos para inferir uma resposta —, o chatbot entregou a síntese da informação em uma média de **30 segundos**.

Isso representa um ganho de eficiência de aproximadamente **400%**. O feedback qualitativo dos participantes corroborou esse dado, destacando a redução da "fadiga cognitiva". Um dos participantes relatou: *"O assistente conseguiu responder de maneira satisfatória... [com] fundamentação e o artigo da norma que referencia a resposta"*.

6.2.2. Eficácia Percebida e Análise de Falhas

No quesito confiabilidade, a percepção dos usuários foi extremamente positiva. Das 50 interações avaliadas (10 alunos $\times$ 5 perguntas):

- **98% (49/50)** das respostas foram classificadas pelos participantes como "Corretas e Úteis".
- **2% (1/50)** foi classificada como "Erro".

Análise do Caso de Erro: Uma análise técnica *post-hoc* foi realizada sobre a única interação classificada como erro (Pergunta: *"Qual nota mínima para aprovação no Exame Final?"*). O chatbot respondeu corretamente citando o Art. 118 ("média final igual ou superior a 6,0"). A classificação negativa por parte do usuário sugere uma confusão conceitual do próprio discente entre "nota do exame" e "média final", evidenciando que a ferramenta manteve o rigor técnico da norma.

6.3. Avaliação de Experiência do Usuário (UX)

Apesar da alta eficácia (acerto) e eficiência (tempo), o feedback qualitativo revelou pontos de atrito na interface. Vários participantes observaram que a exibição dos logs internos de raciocínio do agente (mensagens de *"Tool Call: Supabase_Semantic_Search"*) gerou poluição visual. Um participante observou: *"As tool calls aparecem ao submeter a pergunta... deveria ser direto a resposta"*. Esse feedback indica que, embora a arquitetura *Agentic RAG* funcione perfeitamente no backend, a transparência excessiva do processo de decisão pode confundir usuários finais, sugerindo a necessidade de ocultar esses passos na camada de frontend em versões futuras.

6.4. Discussão dos Resultados

A análise cruzada entre os testes de engenharia e o experimento de campo confirma que a transição da arquitetura V1 (Busca Aproximada) para a V2 (Busca Exaustiva) foi o fator determinante para o sucesso do sistema. A consistência de 100% na recuperação de informação (obtida nos testes internos) e 98% de eficácia percebida (nos testes de campo) corrobora as conclusões de Setty et al. (2024) [Setty et al. 2024], que identificam a qualidade da etapa de recuperação (*Retrieval*) como o principal gargalo de sistemas RAG em domínios especializados.

Adicionalmente, o sistema demonstrou robustez ao lidar com perguntas incompletas ou mal formuladas pelos usuários. Diferente de uma busca tradicional por palavras-chave, que falharia diante de termos coloquiais ou imprecisos, o agente utiliza técnicas de enriquecimento de prompt para expandir a consulta original e inferir a intenção do usuário antes de acessar a base vetorial. Essa abordagem superou limitações observadas em trabalhos correlatos, como o de Silveira Júnior (2023) [Silveira Júnior 2023], garantindo que a linguagem natural informal dos estudantes fosse corretamente mapeada para os termos técnicos da Resolução 177/2012.

Por fim, a escolha de um modelo com comportamento "honesto" (GPT-5-mini), que prioriza a abstenção ao erro, mostrou-se essencial para tarefas intensivas em conhecimento [Lewis et al. 2020], estabelecendo a confiança necessária para a adoção da ferramenta pela comunidade universitária.

7. Conclusão

Este trabalho abordou o desafio de democratizar o acesso à informação normativa dos cursos de graduação da Universidade Federal do Piauí, propondo uma solução baseada em Inteligência Artificial para mitigar a complexidade da Resolução 177/2012. O desenvolvimento de um assistente virtual baseado na arquitetura RAG e orquestrado por agentes inteligentes (*LangGraph*) permitiu alcançar os objetivos propostos de forma satisfatória.

O estudo demonstrou que a principal barreira para a adoção de LLMs em contextos críticos — a alucinação — pode ser efetivamente controlada através de engenharia de software robusta. A implementação de estratégias de **Busca Vetorial Exaustiva** em detrimento de buscas aproximadas, aliada à segmentação semântica da norma por artigos, resultou em um sistema com alta taxa de sucesso na recuperação de contexto. A escolha estratégica do modelo GPT-5-mini priorizou a segurança da informação, garantindo um comportamento "honesto" diante de lacunas de conhecimento.

Os experimentos com usuários validaram a utilidade prática da ferramenta, evidenciando uma redução de tempo de busca de 2 minutos para cerca de 30 segundos por questão, com uma taxa de aprovação de 100% entre os participantes. Conclui-se, portanto, que a aplicação de técnicas de *Agentic RAG* é viável para a modernização do suporte acadêmico, oferecendo uma interface amigável sem renunciar ao rigor jurídico.

Como desdobramentos futuros para a continuidade desta pesquisa, e considerando as limitações deste estudo de caso preliminar, sugere-se:

- **Validação Robusta:** Expandir a validação para um conjunto de testes massivo, criando um gabarito (*Golden Dataset*) que cubra a totalidade dos artigos da Resolução, permitindo a aplicação de métricas estatísticas formais.
- **Testes de Robustez (Fuzzing):** Submeter o assistente a testes de estresse com entradas aleatórias, adversariais ou sem sentido, para avaliar a capacidade do modelo de rejeitar interações fora do escopo.
- **Melhoria de UX:** Refinar a interface para ocultar os logs técnicos (*Tool Calls*), apresentando apenas a resposta final ao usuário para reduzir a carga cognitiva.
- **Integração de Dados:** Conectar o sistema aos dados acadêmicos do aluno (via API do SIGAA), permitindo respostas personalizadas baseadas no histórico escolar do discente.

References

- [Adamopoulou and Moussiades 2020] Adamopoulou, E. and Moussiades, L. (2020). An overview of chatbot technology. In *IFIP international conference on artificial intelligence applications and innovations*, pages 373–383. Springer.
- [Almeida and Xexéo 2019] Almeida, F. and Xexéo, G. (2019). Word embeddings: A survey. *arXiv preprint arXiv:1901.09069*.

- [Altman 1992] Altman, N. S. (1992). An introduction to kernel and nearest-neighbor non-parametric regression. *The American Statistician*, 46(3):175–185.
- [Bentley 1975] Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517.
- [Brasil 2018] Brasil (2018). LEI Nº13.709, DE 14 DE AGOSTO DE 2018 (lei geral de proteção de dados pessoais - lgpd). Diário Oficial da União.
- [Gil 2002] Gil, A. C. (2002). *Como elaborar projetos de pesquisa*. Atlas.
- [Gomaa et al. 2013] Gomaa, W. H., Fahmy, A. A., et al. (2013). A survey of text similarity approaches. *international journal of Computer Applications*, 68(13):13–18.
- [Indyk and Motwani 1998] Indyk, P. and Motwani, R. (1998). Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, pages 604–613.
- [Jurafsky and Martin 2023] Jurafsky, D. and Martin, J. H. (2023). *Speech and Language Processing*. Prentice Hall. 3rd edition.
- [Karpukhin et al. 2020] Karpukhin, V., Oguz, B., Min, S., Lewis, P. S., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pages 6769–6781.
- [Lahitani et al. 2016] Lahitani, A. R., Permanasari, A. E., and Setiawan, N. A. (2016). Cosine similarity to determine similarity measure: Study case in online essay assessment. In *2016 4th International conference on cyber and IT service management*, pages 1–6. IEEE.
- [Lewis et al. 2020] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- [Mikolov et al. 2013] Mikolov, T., Yih, W.-t., and Zweig, G. (2013). Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, pages 746–751.
- [Muja and Lowe 2009] Muja, M. and Lowe, D. G. (2009). Fast approximate nearest neighbors with automatic algorithm configuration. In *VISAPP 2009 - International Conference on Computer Vision Theory and Applications*, pages 331–340.
- [Rajpurkar et al. 2016] Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. (2016). Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*.
- [Reiter and Dale 1997] Reiter, E. and Dale, R. (1997). Building applied natural language generation systems. *Natural language engineering*, 3(1):57–87.
- [Semaan 2012] Semaan, P. (2012). Natural language generation: an overview. *J Comput Sci Res*, 1(3):50–57.

- [Setty et al. 2024] Setty, S., Thakkar, H., Lee, A., Chung, E., and Vidra, N. (2024). Improving retrieval for rag based question answering models on financial documents. *arXiv preprint arXiv:2404.07221*.
- [Silveira Júnior 2023] Silveira Júnior, C. A. C. (2023). Chatbot de perguntas e respostas para o regimento geral da ufpe utilizando chatgpt e busca semântica. Master’s thesis, UNIVERSIDADE FEDERAL DE PELOTAS.
- [Turian et al. 2010] Turian, J., Ratinov, L., and Bengio, Y. (2010). Word representations: a simple and general method for semi-supervised learning. In *Proceedings of the 48th annual meeting of the association for computational linguistics*, pages 384–394.
- [Turing 2007] Turing, A. M. (2007). Computing machinery and intelligence. In *Parsing the Turing test: Philosophical and methodological issues in the quest for the thinking computer*, pages 23–65. Springer.
- [Vaswani et al. 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- [Weizenbaum 1966] Weizenbaum, J. (1966). Eliza—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1):36–45.
- [Wohlin et al. 2012] Wohlin, C. et al. (2012). *Experimentation in software engineering*. Springer.
- [Zhang et al. 2019] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., and Artzi, Y. (2019). Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*.